# L08 — Bubble Sort: Algorithm and Complexity Analysis

**Unit:** 1 | **Chapter:** Sorting Techniques | **BT Level:** BT4 | **CO:** CO1, CO5

---

## Learning Objectives

- Implement bubble sort and its optimized variant
- Trace through the algorithm step by step
- Derive the time complexity for best, worst, and average cases
- Explain why bubble sort is rarely used in production

---

## 1. Concept of Bubble Sort

**Bubble Sort** repeatedly steps through the list, compares adjacent elements, and swaps them if they are in the wrong order. The largest (or smallest) element "bubbles up" to its correct position after each pass.

**Key Insight:** After k passes, the k largest elements are in their correct final positions.

---

## 2. Algorithm

### 2.1 Basic Bubble Sort

```
def bubble_sort(arr):
    n = len(arr)
    for i in range(n - 1):          # n-1 passes
        for j in range(n - 1 - i):  # Each pass shrinks by 1
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]  # Swap
    return arr
```

### 2.2 Optimized Bubble Sort (with early termination)

```
def bubble_sort_optimized(arr):
    n = len(arr)
    for i in range(n - 1):
        swapped = False
        for j in range(n - 1 - i):
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
                swapped = True
        if not swapped:      # No swap in this pass → already sorted
            break
    return arr
```

The `swapped` flag makes the algorithm **adaptive**: if no swap occurs in a pass, the array is already sorted.

---

# 3. Detailed Trace

Array: `[5, 3, 8, 4, 2]`, n = 5

**Pass 1 (i = 0): j goes from 0 to 3**

| j | Compare | Swap? | Array |
|---|---|---|---|
| 0 | 5 > 3 | Yes | [**3, 5**, 8, 4, 2] |
| 1 | 5 > 8 | No | [3, 5, 8, 4, 2] |
| 2 | 8 > 4 | Yes | [3, 5, **4, 8**, 2] |
| 3 | 8 > 2 | Yes | [3, 5, 4, **2, 8**] |

After pass 1: largest element (8) is at last position.

**Pass 2 (i = 1): j goes from 0 to 2**

| j | Compare | Swap? | Array |
|---|---|---|---|
| 0 | 3 > 5 | No | [3, 5, 4, 2, 8] |
| 1 | 5 > 4 | Yes | [3, **4, 5**, 2, 8] |
| 2 | 5 > 2 | Yes | [3, 4, **2, 5**, 8] |

**Pass 3 (i = 2): j goes from 0 to 1**

| j | Compare | Swap? | Array |
|---|---|---|---|
| 0 | 3 > 4 | No | [3, 4, 2, 5, 8] |
| 1 | 4 > 2 | Yes | [3, **2, 4**, 5, 8] |

**Pass 4 (i = 3): j goes from 0 to 0**

| j | Compare | Swap? | Array |
|---|---|---|---|
| 0 | 3 > 2 | Yes | [**2, 3**, 4, 5, 8] |

**Final:** `[2, 3, 4, 5, 8]` ✓

---

# 4. Complexity Analysis

## 4.1 Number of Comparisons

In pass i (0-indexed), the inner loop runs (n-1-i) times.

Total comparisons = $\sum_{i=0}^{n-2}(n-1-i) = (n-1) + (n-2) + \ldots + 1 = \frac{n(n-1)}{2}$

## 4.2 Case Analysis

| Case | Condition | Swaps | Comparisons | Time |
|---|---|---|---|---|
| **Best** | Already sorted | 0 | n-1 (with optimization) | **O(n)** |
| **Best** | Already sorted | 0 | n(n-1)/2 (without optimization) | $O(n^2)$ |
| **Worst** | Reverse sorted | n(n-1)/2 | n(n-1)/2 | $O(n^2)$ |
| **Average** | Random order | $\sim n^2/4$ | n(n-1)/2 | $O(n^2)$ |

### 4.3 Space Complexity

O(1) — only a constant number of extra variables (i, j, temp).

---

# 5. Properties of Bubble Sort

| Property | Value |
|---|---|
| Time (best) | O(n) with optimization |
| Time (worst/average) | $O(n^2)$ |
| Space | O(1) |
| In-place | Yes |
| Stable | Yes |
| Adaptive | Yes (with flag) |

**Stable:** Equal elements are never swapped (only swap when `arr[j] > arr[j+1]`, not >=).

---

# 6. Worked Complexity Example

Worst case: reverse sorted array `[5, 4, 3, 2, 1]`

Pass 1: 4 comparisons, 4 swaps → [4, 3, 2, 1, **5**]
Pass 2: 3 comparisons, 3 swaps → [3, 2, 1, **4**, 5]
Pass 3: 2 comparisons, 2 swaps → [2, 1, **3**, 4, 5]
Pass 4: 1 comparison, 1 swap → [**1**, 2, 3, 4, 5]

Total comparisons: 4 + 3 + 2 + 1 = 10 = 5(5-1)/2 = n(n-1)/2 ✓

---

# 7. Why Bubble Sort is Rarely Used in Practice

- $O(n^2)$ on average/worst — much worse than O(n log n) for large n
- Even for small arrays, insertion sort is faster in practice (fewer writes)
- However, bubble sort is:
  - Easy to implement and understand
  - Good for teaching comparison-based sorting concepts
  - Useful for detecting whether an array is already sorted (O(n) with flag)

---

# 8. Variants

### Cocktail Shaker Sort (Bidirectional Bubble Sort)

Traverses the array in both directions alternately — slightly better in practice for partially sorted arrays:

```
def cocktail_sort(arr):
    n = len(arr)
    swapped = True
    start, end = 0, n - 1
    while swapped:
        swapped = False
        for i in range(start, end):
            if arr[i] > arr[i + 1]:
                arr[i], arr[i + 1] = arr[i + 1], arr[i]
                swapped = True
        end -= 1
        for i in range(end - 1, start - 1, -1):
            if arr[i] > arr[i + 1]:
                arr[i], arr[i + 1] = arr[i + 1], arr[i]
                swapped = True
        start += 1
    return arr
```

---

# Summary

- Bubble sort repeatedly swaps adjacent elements to push the largest unsorted element to its position.
- Worst and average case: **O($n^2$)**; with the `swapped` flag, best case is **O(n)**.
- Stable (preserves relative order of equal elements), in-place (O(1) extra space).
- Inefficient for large datasets — used primarily for educational purposes.

---

# References


- Cormen et al., *Introduction to Algorithms*, Problem 2.2 (MIT Press, 2022)
- Skiena, *The Algorithm Design Manual*, Ch. 4.2 (Springer, 2020)